JAVA / 阶段 01

面向对象程序设计

阶段一 · 答案

对应原题库全部 30 道题

完整代码 / 输入输出 / 基础思路

使用方式

每道题保存为独立的 Main.java。先读题并自己尝试，再对照答案运行；不要把不同题的 Main 类放进同一个文件。

命令：javac -encoding UTF-8 Main.java；随后执行 java Main。输入样例中的数据后按回车。网站可直接编辑、编译和运行。

解法使用变量、判断、循环、数组、方法和基础面向对象知识，不依赖第三方 Java 库。

题目口径与说明

1. 依据用户提供的阶段一题库逐题整理，共30个原编号；每题是独立、无包名的 Main.java，使用JDK17基础语法。
2. 代码为学习练习参考，建议先独立尝试再对照。样例中的标准输入不包含解释文字，固定数据题输入为空。
3. 第14与15条在原文中属于同一题及其示例，为对应原编号保留两条；统一输入顺序为 n a。
4. 第24题修正了原文不合法的数组初始化语法及中文逗号；第27题兼答括注的最小值，行列号从1开始。
5. 第10题不一定存在三位约数，已补充不存在的输出；第19题空数据平均值按程序约定显示0.0；第30题数量允许0，额外要求每种至少1只时只有3组。
6. 第21题Fibonacci从1、1开始；第6题按逆序字符保留末尾0；第14/15题用long时限制项数不超过18。
7. 普通double小数运算存在表示误差；long、int都有上限，不应任意扩大输入范围后仍假设结果精确。

目录

第 01 题 三个整数从大到小排列 4

第 02 题 分段函数 4

第 03 题 switch 四则运算 5

第 04 题 交错奇数求和 7

第 05 题 分数数列前30项和 7

第 06 题 位数与数字逆序 8

第 07 题 调用方法判断水仙花数 9

第 08 题 统计字符串中的数字字符 10

第 09 题 被2或3整除的数之和 11

第 10 题 最大的三位数约数 11

第 11 题 n个整数中的最大值 12

第 12 题 闭区间整数求和 13

第 13 题 四个数字组成不重复三位数 14

第 14 题 重复数字数列求和 15

第 15 题 原题第14题的示例（保留编号） 16

第 16 题 最大公约数和最小公倍数 17

第 17 题 m以内的水仙花数 18

第 18 题 阶乘方法与求和 19

第 19 题 输入0结束并求平均 19

第 20 题 数组原地逆序 20

第 21 题 数组保存Fibonacci前30项 21

第 22 题 20个数的最小值 22

第 23 题 方法返回课程平均分 23

第 24 题 冒泡排序升序 24

第 25 题 日期是当年第几天 25

第 26 题 有序数组二分查找 26

第 27 题 矩阵最大最小值及位置 27

第 28 题 数组顺序查找 28

第 29 题 四位回文数 29

第 30 题 百钱百鸡枚举 30

复习与参考 31

第 01 题 三个整数从大到小排列

题目

从键盘输入三个整数 a、b、c，按从大到小的顺序输出。

基础思路

先把较大的数交换到 a，再把剩余两个数中较大的数交换到 b。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();
        int temp;
        if (a < b) { temp = a; a = b; b = temp; }
        if (a < c) { temp = a; a = c; c = temp; }
        if (b < c) { temp = b; b = c; c = temp; }
        System.out.println(a + " " + b + " " + c);
    }
}
```

样例输入

```
3 9 5
```

样例输出

```
9 5 3
```

注意

比较用 < 或 >，赋值用 =；它们的作用不同。

第 02 题 分段函数

题目

输入实数 x，计算 f(x)：x <= 0 时为 $x^3 + 5$；0 < x <= 20 时为 $x^2 - x + 3$；x > 20 时为 7x + 2。输出保留两位小数。

基础思路

按区间顺序判断 x，只执行对应的一条计算公式。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double x = sc.nextDouble();
        double y;
        if (x <= 0) {
            y = x * x * x + 5;
        } else if (x <= 20) {
            y = x * x - x + 3;
        } else {
            y = 7 * x + 2;
        }
        System.out.printf(java.util.Locale.US, "%.2f%n", y);
    }
}
```

样例输入

```
3
```

样例输出

```
9.00
```

注意

不能直接写 0 < x <= 20；Java 中应写 x > 0 && x <= 20，或采用本题的分支顺序。

第 03 题 switch 四则运算

题目

输入一个简单四则表达式，例如 12-8，使用 Scanner 和 switch 输出表达式及计算结果。本答案允许一个二元运算、普通小数、操作数前的正负号和空格，不处理括号、科学计数法或连续多个运算。

基础思路

先找运算符，把左右两部分转成数，再用 switch 执行对应运算。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String expression = sc.nextLine().replace(" ", "");
        int position = -1;
        for (int i = 1; i < expression.length(); i++) {
            char ch = expression.charAt(i);
            if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
                position = i;
```

Main.java（续）

```
                break;
            }
        }
        if (position == -1) {
            System.out.println("表达式格式错误");
            return;
        }
        double a = Double.parseDouble(expression.substring(0, position));
        double b = Double.parseDouble(expression.substring(position + 1));
        char op = expression.charAt(position);
        double result = 0;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/':
                if (b == 0) {
                    System.out.println("除数不能为0");
                    return;
                }
                result = a / b;
                break;
            default:
                System.out.println("不支持的运算符");
                return;
        }
        String answer;
        if (result >= Long.MIN_VALUE && result < Long.MAX_VALUE
                && result == (long) result) {
            answer = String.valueOf((long) result);
        } else {
            answer = String.valueOf(result);
        }
        System.out.println(expression + "=" + answer);
    }
}
```

样例输入

```
12-8
```

样例输出

```
12-8=4
```

注意

case 后通常需要 break；漏掉会继续执行后续 case。

第 04 题 交错奇数求和

题目

计算 1-3+5-7+…-99+101 的值。

基础思路

用 i 依次取所有奇数，每加完一项便把符号乘以 -1。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        int sign = 1;
        for (int i = 1; i <= 101; i += 2) {
            sum = sum + sign * i;
            sign = -sign;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
51
```

注意

i++ 会遍历所有整数，不能代替 i+=2。

第 05 题 分数数列前30项和

题目

求 2/1、3/2、5/3、8/5、13/8、21/13……的前 30 项之和，结果保留六位小数。

基础思路

分子与分母都来自相邻的 Fibonacci 数：新分子等于旧分子加旧分母，新分母等于旧分子。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        double numerator = 2;
```

Main.java（续）

```
        double denominator = 1;
        double sum = 0;
        for (int i = 1; i <= 30; i++) {
            sum = sum + numerator / denominator;
            double next = numerator + denominator;
            denominator = numerator;
            numerator = next;
        }
        System.out.printf(java.util.Locale.US, "%.6f%n", sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
48.840601
```

注意

不能先写 numerator=numerator+denominator，再写 denominator=numerator，否则分母取到的是新分子。

第 06 题 位数与数字逆序

题目

输入一个正整数 k，输出它的位数，并按逆序输出各位数字。本答案用 long 保存 k；逆序按字符显示，因此 1200 显示为 0021。第一行是位数，第二行是逆序结果。

基础思路

反复用 %10 取出个位，用 /10 去掉个位，同时计数并拼接。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long k = sc.nextLong();
        if (k <= 0) {
            System.out.println("请输入正整数");
            return;
        }
        long temp = k;
        int count = 0;
        String reverse = "";
        while (temp > 0) {
            reverse = reverse + temp % 10;
            count++;
```

Main.java（续）

```
            temp = temp / 10;
        }
        System.out.println(count);
        System.out.println(reverse);
    }
}
```

样例输入

```
123
```

样例输出

```
3
321
```

注意

% 是取余，/ 对整数执行整除，两者不能混淆。

第 07 题 调用方法判断水仙花数

题目

输出所有三位水仙花数，并定义、调用 is(number) 判断一个数的各位数字立方和是否等于自身。

基础思路

主方法枚举 100 到 999；is 方法只负责检查一个数是否符合定义。

完整参考代码 · Main.java

```
public class Main {
    public static boolean is(int number) {
        if (number < 100 || number > 999) return false;
        int a = number / 100;
        int b = number / 10 % 10;
        int c = number % 10;
        return a * a * a + b * b * b + c * c * c == number;
    }
    public static void main(String[] args) {
        boolean first = true;
        for (int i = 100; i <= 999; i++) {
            if (is(i)) {
                if (!first) System.out.print(" ");
                System.out.print(i);
                first = false;
            }
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
153 370 371 407
```

注意

方法名按题目使用 is，不要把它误写为关键字 if。

第 08 题 统计字符串中的数字字符

题目

输入一整行字符串，统计其中数字字符 0 到 9 的个数。空格也属于这行字符串的一部分。

基础思路

从左到右检查每个字符，落在字符 0 到 9 范围内就把计数加一。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String text = sc.nextLine();
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch >= '0' && ch <= '9') count++;
        }
        System.out.println(count);
    }
}
```

样例输入

```
Java 2026, score=95
```

样例输出

```
6
```

注意

循环条件应为 i<text.length()，不能写 <=，否则最后会访问越界。

第 09 题 被2或3整除的数之和

题目

输入正整数 n，求 1 到 n 中所有能够被 2 或者被 3 整除的数之和。

基础思路

枚举每个整数，只要满足两个整除条件之一，就累加一次。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("请输入正整数");
            return;
        }
        long sum = 0;
        for (long i = 1; i <= n; i++) {
            if (i % 2 == 0 || i % 3 == 0) sum += i;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
10
```

样例输出

```
42
```

注意

题目是“或者”，应使用 ||；&& 会变成同时能被 2 和 3 整除。

第 10 题 最大的三位数约数

题目

用户输入 n，要求 n>10000；求 n 的约数中最大的三位数。如果不存在三位数约数，明确输出“不存在三位数约数”。

基础思路

从 999 倒着检查到 100，找到第一个能整除 n 的数就结束。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        if (n <= 10000) {
            System.out.println("n必须大于10000");
            return;
        }
        for (int i = 999; i >= 100; i--) {
            if (n % i == 0) {
                System.out.println(i);
                return;
            }
        }
        System.out.println("不存在三位数约数");
    }
}
```

样例输入

```
12345
```

样例输出

```
823
```

注意

从 100 向上找到就立即结束，只会得到最小的三位数约数。

第 11 题 n个整数中的最大值

题目

先输入整数个数 n，再输入 n 个整数，输出其中最大值，格式为 max=数值。n 必须大于 0。

基础思路

把第一个实际输入值当成当前最大值，再依次比较其余 n-1 个数。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("n必须大于0");
            return;
        }
```

Main.java（续）

```
        int max = sc.nextInt();
        for (int i = 1; i < n; i++) {
            int value = sc.nextInt();
            if (value > max) max = value;
        }
        System.out.println("max=" + max);
    }
}
```

样例输入

```
5
3 9 -2 7 5
```

样例输出

```
max=9
```

注意

不要把 n 也拿去和后面的数据比较。

第 12 题 闭区间整数求和

题目

输入两个整数 m、n，要求 m<=n；求从 m 到 n 的所有整数之和，包含两个端点，输出 S=结果。

基础思路

从 m 开始逐个累加到 n，使用 long 保存总和。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt();
        int n = sc.nextInt();
        if (m > n) {
            System.out.println("m不能大于n");
            return;
        }
        long sum = 0;
        for (long i = m; i <= n; i++) sum += i;
        System.out.println("S=" + sum);
    }
}
```

样例输入

```
3 5
```

样例输出

```
S=12
```

注意

若条件误写成 i<n，会漏掉 n。

第 13 题 四个数字组成不重复三位数

题目

用 1、2、3、4 组成互不相同且每个数内部无重复数字的三位数，输出全部三位数和总个数。

基础思路

百、十、个位分别枚举 1—4，只有三个数字两两不同才输出。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int a = 1; a <= 4; a++) {
            for (int b = 1; b <= 4; b++) {
                for (int c = 1; c <= 4; c++) {
                    if (a != b && a != c && b != c) {
                        if (count > 0) System.out.print(" ");
                        System.out.print(100 * a + 10 * b + c);
                        count++;
                    }
                }
            }
        }
        System.out.println();
        System.out.println("总数=" + count);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
123 124 132 134 142 143 213 214 231 234 241 243 312 314 321 324 341 342 412 413 421 423 431 432
总数=24
```

注意

只检查 a!=b 和 b!=c 会漏掉 a=c 的重复情况。

第 14 题 重复数字数列求和

题目

计算 Sn=a+aa+aaa+…（共 n 项）。按原题示例统一输入顺序为 n a。为使用基础 long 且不溢出，本答案约定 1<=n<=18、0<=a<=9。

基础思路

每一项由上一项乘 10 再加 a 得到，随后加入总和。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a = sc.nextInt();
        if (n < 1 || n > 18 || a < 0 || a > 9) {
            System.out.println("要求1<=n<=18且0<=a<=9");
            return;
        }
        long term = 0;
        long sum = 0;
        for (int i = 1; i <= n; i++) {
            term = term * 10 + a;
            sum = sum + term;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
3 2
```

样例输出

```
246
```

注意

表达式 aa 不是 Java 变量 a 的平方，也不是 a+a，应通过 term*10+a 构造。

第 15 题 原题第14题的示例（保留编号）

题目

原文第 15 条是第 14 题续写的示例：S5=2+22+222+2222+22222。继续采用输入 n a；原文样例 3 2 的结果为 246。这里保留独立编号方便对照，算法与第 14 题相同。

基础思路

仍用 term=term*10+a 逐项生成，以 n 控制项数；数字 2 由输入提供。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a = sc.nextInt();
        if (n < 1 || n > 18 || a < 0 || a > 9) {
            System.out.println("要求1<=n<=18且0<=a<=9");
            return;
        }
        long term = 0;
        long sum = 0;
        for (int i = 1; i <= n; i++) {
            term = term * 10 + a;
            sum = sum + term;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
5 2
```

样例输出

```
24690
```

注意

原文同时出现“n=5”的讲解与“3 2”的输入样例，二者属于不同示例，输出应分别是 24690 和 246。

第 16 题 最大公约数和最小公倍数

题目

输入两个整数，由主方法分别调用两个方法求最大公约数与最小公倍数。第一行输出最大公约数，第二行输出最小公倍数。支持负数和单个 0；两个数同时为 0 时提示无定义。

基础思路

用辗转相除法求 gcd，再用 |a/gcd*b| 求 lcm，先除后乘降低溢出风险。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static long gcd(long a, long b) {
        a = Math.abs(a);
        b = Math.abs(b);
        while (b != 0) {
            long remainder = a % b;
            a = b;
            b = remainder;
        }
        return a;
    }
    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        return Math.abs(a / gcd(a, b) * b);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long a = sc.nextInt();
        long b = sc.nextInt();
        if (a == 0 && b == 0) {
            System.out.println("两个0的最大公约数无定义");
            return;
        }
        System.out.println(gcd(a, b));
        System.out.println(lcm(a, b));
    }
}
```

样例输入

```
12 18
```

样例输出

```
6
36
```

注意

每次计算余数必须在覆盖 a 和 b 之前完成。

第 17 题 m以内的水仙花数

题目

输入 m（0<=m<=1000），输出不超过 m 的所有三位水仙花数。没有符合条件的数时输出“无”。

基础思路

从 100 枚举到 m，同时限制不超过 999，复用三位数拆分判断方法。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static boolean is(int number) {
        int a = number / 100;
        int b = number / 10 % 10;
        int c = number % 10;
        return a * a * a + b * b * b + c * c * c == number;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt();
        if (m < 0 || m > 1000) {
            System.out.println("要求0<=m<=1000");
            return;
        }
        boolean found = false;
        for (int i = 100; i <= m && i <= 999; i++) {
            if (is(i)) {
                if (found) System.out.print(" ");
                System.out.print(i);
                found = true;
            }
        }
        if (!found) System.out.print("无");
        System.out.println();
    }
}
```

样例输入

```
1000
```

样例输出

```
153 370 371 407
```

注意

题目中的水仙花数只指三位数，不应把 1 之类的一位数列入。

第 18 题 阶乘方法与求和

题目

编写 fact(n) 计算 n!，调用该方法计算并输出 4!+7!+8!。

基础思路

fact 使用循环从 1 乘到 n，主方法调用三次并相加。

完整参考代码 · Main.java

```
public class Main {
    public static long fact(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) result *= i;
        return result;
    }
    public static void main(String[] args) {
        long sum = fact(4) + fact(7) + fact(8);
        System.out.println(sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
45384
```

注意

Java 中 ！是逻辑非，不能直接写 4! 来求阶乘。

第 19 题 输入0结束并求平均

题目

输入一组数，读到 0 时结束；0 不计入数据。输出个数和平均值，平均值保留一位小数。若第一项就是 0，约定输出 0 0.0，避免除以 0。

基础思路

读入后先判断是否为结束标记，再计数和累加，最后计算平均值。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int count = 0;
        double sum = 0;
        while (sc.hasNextDouble()) {
            double value = sc.nextDouble();
            if (value == 0) break;
            count++;
            sum += value;
        }
        double average = 0;
        if (count > 0) average = sum / count;
        System.out.printf(java.util.Locale.US, "%d %.1f%n", count, average);
    }
}
```

样例输入

```
5 10 15 20 25 30 35 40 45 50 0
```

样例输出

```
10 27.5
```

注意

先 count++ 再判断 0，会把结束标记错误统计进去。

第 20 题 数组原地逆序

题目

把数组 {8,6,5,4,1} 的值按逆序重新存放，再输出结果。注意要求改变数组内部顺序。

基础思路

交换首尾，再交换次首与次尾，只处理数组前半部分。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] a = {8, 6, 5, 4, 1};
        for (int i = 0; i < a.length / 2; i++) {
            int j = a.length - 1 - i;
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        for (int i = 0; i < a.length; i++) {
```

Main.java（续）

```
            if (i > 0) System.out.print(" ");
            System.out.print(a[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 4 5 6 8
```

注意

最后一个元素的下标是 length-1，不是 length。

第 21 题 数组保存Fibonacci前30项

题目

用数组保存并输出 Fibonacci 数列前 30 项。本答案采用第 1 项为 1、第 2 项为 1 的约定。

基础思路

先保存两个初值，从下标 2 开始，让当前项等于前两项之和。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] f = new int[30];
        f[0] = 1;
        f[1] = 1;
        for (int i = 2; i < f.length; i++) {
            f[i] = f[i - 1] + f[i - 2];
        }
        for (int i = 0; i < f.length; i++) {
            if (i > 0) System.out.print(" ");
            System.out.print(f[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75025 121393
    196418 317811 514229 832040
```

注意

循环不能从 i=0 就访问 f[i-2]，那样会出现负下标。

第 22 题 20个数的最小值

题目

从键盘输入 20 个数存入数组，然后输出最小值。本答案使用 double，允许输入普通整数和小数。

基础思路

先读满数组，再用第一个元素初始化 min，依次比较剩余元素。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double[] a = new double[20];
        for (int i = 0; i < a.length; i++) a[i] = sc.nextDouble();
        double min = a[0];
        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) min = a[i];
        }
        System.out.println(min);
    }
}
```

样例输入

```
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

样例输出

```
1.0
```

注意

本题固定输入 20 个数据，开头不需要额外输入一个 n。

第 23 题 方法返回课程平均分

题目

编写 fun 方法计算 n 门课程的平均分，并作为返回值返回。输入先给 n，再给 n 个成绩，输出平均分并保留两位小数。

基础思路

主方法读入成绩数组，把数组交给 fun；fun 求和后除以数组长度。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static double fun(double[] scores) {
        double sum = 0;
        for (int i = 0; i < scores.length; i++) sum += scores[i];
        return sum / scores.length;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("课程数必须大于0");
            return;
        }
        double[] scores = new double[n];
        for (int i = 0; i < n; i++) scores[i] = sc.nextDouble();
        double average = fun(scores);
        System.out.printf(java.util.Locale.US, "%.2f%n", average);
    }
}
```

样例输入

```
3
80 90 100
```

样例输出

```
90.00
```

注意

返回类型必须是 double，不能声明为 void 后又 return 一个数。

第 24 题 冒泡排序升序

题目

将数组 {5,4,3,2,1} 用冒泡排序升序排列后输出。原文 int a[]=new int[5]{…} 不是合法 Java 初始化语法；本答案改为 int[] a={5,4,3,2,1}。

基础思路

每趟比较相邻元素，左边较大就交换，一趟结束把当前最大值放到未排序部分的末尾。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 4, 3, 2, 1};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }
        for (int i = 0; i < a.length; i++) {
            if (i > 0) System.out.print(" ");
            System.out.print(a[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 2 3 4 5
```

注意

只做一趟只能保证最大的数到最后，不能保证全部有序。

第 25 题 日期是当年第几天

题目

输入年、月、日，输出它是该年的第几天。按公历闰年规则处理，非法日期输出“日期不合法”。

基础思路

先确定各月天数，再把该月之前各月的天数加上当前日。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int year = sc.nextInt();
        int month = sc.nextInt();
        int day = sc.nextInt();
        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        boolean leap = year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);
        if (leap) days[1] = 29;
        if (year <= 0 || month < 1 || month > 12) {
            System.out.println("日期不合法");
            return;
        }
        if (day < 1 || day > days[month - 1]) {
            System.out.println("日期不合法");
            return;
        }
        int total = day;
        for (int i = 0; i < month - 1; i++) total += days[i];
        System.out.println(total);
    }
}
```

样例输入

```
2024 3 1
```

样例输出

```
61
```

注意

“能被 4 整除就是闰年”会错误处理 1900。

第 26 题 有序数组二分查找

题目

编写方法，在已按从小到大排列的 n 个元素数组中二分查找 x，找到返回下标，否则返回 -1。输入 n，再输入 n 个有序整数，最后输入 x；下标从 0 开始。

基础思路

维护左右端点，比较中间元素，每次排除不可能的一半区间。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static int binarySearch(int[] a, int x) {
        int left = 0;
        int right = a.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (a[mid] == x) return mid;
            if (a[mid] < x) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 0) {
            System.out.println("n不能小于0");
            return;
        }
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int x = sc.nextInt();
        System.out.println(binarySearch(a, x));
    }
}
```

样例输入

```
5
1 3 5 7 9
7
```

样例输出

```
3
```

注意

有序是必需条件；不要把数组排序后又把新下标当作原数组下标。

第 27 题 矩阵最大最小值及位置

题目

输入一个 3×4 整数矩阵。原题正文要求最大值，末尾括号又写“最小值”，故同时给出最大值、最小值及各自所在行列。行列从 1 开始；重复极值报告按行扫描遇到的第一个位置。

基础思路

将第一个元素作为最大最小初值，逐行扫描，同时维护数值和所在下标。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] a = new int[3][4];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) a[i][j] = sc.nextInt();
        }
        int max = a[0][0], min = a[0][0];
        int maxRow = 0, maxCol = 0, minRow = 0, minCol = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                if (a[i][j] > max) {
                    max = a[i][j]; maxRow = i; maxCol = j;
                }
                if (a[i][j] < min) {
                    min = a[i][j]; minRow = i; minCol = j;
                }
            }
        }
        System.out.println("最大值=" + max + " 行=" + (maxRow + 1) + " 列=" + (maxCol + 1));
        System.out.println("最小值=" + min + " 行=" + (minRow + 1) + " 列=" + (minCol + 1));
    }
}
```

样例输入

```
1 2 3 4
5 6 7 8
9 10 11 12
```

样例输出

```
最大值=12 行=3 列=4
最小值=1 行=1 列=1
```

注意

不要把 a[i][j] 的行列顺序写反。

第 28 题 数组顺序查找

题目

编写方法，在 n 个元素的数组中查找 x；找到返回下标，否则返回 -1。输入 n、n 个整数、x。数组不必有序；若重复出现，返回第一次出现的下标。

基础思路

从下标 0 开始逐个比较，遇到相等元素立即返回；全部检查后仍未找到则返回 -1。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static int search(int[] a, int x) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == x) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 0) {
            System.out.println("n不能小于0");
            return;
        }
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int x = sc.nextInt();
        System.out.println(search(a, x));
    }
}
```

样例输入

```
5
8 3 9 4 1
4
```

样例输出

```
3
```

注意

在 if 的 else 中直接 return -1 会只比较第一个元素。

第 29 题 四位回文数

题目

输入一个四位正整数，判断它是不是回文数，例如 1221 是，1234 不是。不是 1000—9999 的输入输出“请输入四位正整数”。

基础思路

四位数只需判断千位等于个位，同时百位等于十位。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 1000 || n > 9999) {
            System.out.println("请输入四位正整数");
            return;
        }
        int thousands = n / 1000;
        int hundreds = n / 100 % 10;
        int tens = n / 10 % 10;
        int units = n % 10;
        if (thousands == units && hundreds == tens) {
            System.out.println("是回文数");
        } else {
            System.out.println("不是回文数");
        }
    }
}
```

样例输入

```
1221
```

样例输出

```
是回文数
```

注意

只比较千位与个位不够，例如 1231 并不是回文。

第 30 题 百钱百鸡枚举

题目

小鸡 1 元买 3 只，公鸡 3 元一只，母鸡 5 元一只。用 100 元恰好买 100 只鸡，列出所有方案。各类数量允许为 0；若另要求每种都必须购买，应排除母鸡为 0 的方案。

基础思路

枚举公鸡和母鸡数量，用总只数确定小鸡，再验证小鸡能否三只成组以及总价是否为100。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int rooster = 0; rooster <= 100 / 3; rooster++) {
            for (int hen = 0; hen <= 100 / 5; hen++) {
                int chick = 100 - rooster - hen;
                if (chick >= 0 && chick % 3 == 0
                        && 3 * rooster + 5 * hen + chick / 3 == 100) {
                    System.out.println("公鸡=" + rooster + " 母鸡=" + hen + " 小鸡=" + chick);
                    count++;
                }
            }
        }
        System.out.println("方案数=" + count);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
公鸡=4 母鸡=12 小鸡=84
公鸡=11 母鸡=8 小鸡=81
公鸡=18 母鸡=4 小鸡=78
公鸡=25 母鸡=0 小鸡=75
方案数=4
```

注意

整数表达式 1/3 等于 0，不能写 chick*(1/3) 计算费用。

复习与参考

复习时先做没有输入的题，再做单次输入，最后做循环输入和多个对象的题。遇到相同类名，回到对应题目录单独编译。

原题库：用户提供的阶段一、阶段二题库 PDF。原题中的重复编号、示例编号和歧义已经在封面说明或对应题目中标出。

Oracle Java Tutorials：语言基础

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

Oracle Java Tutorials：面向对象概念

https://docs.oracle.com/javase/tutorial/java/concepts/index.html

本文使用 JDK 17 可运行的基础语法；上面的概念教程版本较早，变量、循环、类与继承等基础仍适用。